



LeetCode 905: Sort Array By Parity

1. Problem Title & Link

Title: LeetCode 905: Sort Array By Parity

Link: <https://leetcode.com/problems/sort-array-by-parity/>

2. Problem Statement (Short Summary)

Given an integer array `nums`, rearrange the array so that:

- All **even numbers** appear first
- All **odd numbers** appear after them

Return the resulting array.

The relative order does **not** matter.

Example

Input:

`nums = [3,1,2,4]`

Output:

`[2,4,3,1]`

Other valid answers:

`[4,2,1,3]`

`[2,4,1,3]`

Because all even numbers come before odd numbers.

3. Examples (Input → Output)

Example 1

Input:

`[3,1,2,4]`

Output:

`[2,4,3,1]`

Example 2

Input:

`[0]`

Output:

`[0]`

Example 3



Input:

[2,4,6,8]

Output:

[2,4,6,8]

Already sorted by parity.

Example 4

Input:

[1,3,5,7]

Output:

[1,3,5,7]

No even numbers exist.

4. Constraints

$1 \leq \text{nums.length} \leq 5000$

$0 \leq \text{nums}[i] \leq 5000$

Important:

Need evens before odds.

Order inside groups does not matter.

5. Core Concept (Pattern / Topic)

Two Pointers

Secondary Topics:

- Array Partitioning
- In-Place Swapping

6. Thought Process (Step-by-Step Explanation)

Brute Force Idea

Create two lists.

even = []

odd = []

Traverse array.

Put numbers into corresponding lists.

Combine:



even + odd

Works.

Complexity

Time : $O(n)$

Space : $O(n)$

Uses extra memory.

Better Observation

We don't care about exact order.

We only care:

Even numbers first

Odd numbers later

This is a partition problem.

Similar To

Quick Sort Partition

Two Pointer Idea

Use:

left -> from beginning

right -> from end

Rules

If left points to even:

Correct position

Move left

If right points to odd:

Correct position

Move right



If:

left -> odd

right -> even

Swap.

Example

Input:

[3,1,2,4]

Initial

L R

3 1 2 4

Odd on left.

Even on right.

Swap.

4 1 2 3

Move both.

Now

4 1 2 3

L R

Swap.

4 2 1 3

Done.

7. Visual / Intuition Diagram (ASCII Diagram)

```
Input:
[3,1,2,4]
Initial:
L       R

3 1 2 4
Swap:
4 1 2 3
```



Move inward:

L R

4 1 2 3

Swap:

4 2 1 3

Result:

Evens | Odds

4 2 | 1 3

8. Pseudocode (Language Independent)

```
left = 0

right = n-1

while left < right

    if nums[left] is even

        left++

    else if nums[right] is odd

        right--

    else

        swap

return nums
```

9. Code Implementation

Python

```
class Solution:
    def sortArrayByParity(self, nums):

        left = 0
        right = len(nums) - 1

        while left < right:

            if nums[left] % 2 == 0:

                left += 1

            elif nums[right] % 2 == 1:
```



```
        right -= 1

    else:

        nums[left], nums[right] = \
            nums[right], nums[left]

        left += 1
        right -= 1

    return nums
```

Java

```
class Solution {

    public int[] sortArrayByParity(int[] nums) {

        int left = 0;
        int right = nums.length - 1;

        while(left < right) {

            if(nums[left] % 2 == 0) {

                left++;

            } else if(nums[right] % 2 == 1) {

                right--;

            } else {

                int temp = nums[left];
                nums[left] = nums[right];
                nums[right] = temp;

                left++;
                right--;

            }

        }

        return nums;

    }

}
```

10. Time & Space Complexity

Time Complexity



$O(n)$

Each element processed at most once.

Space Complexity

$O(1)$

In-place solution.

11. Common Mistakes / Edge Cases

Mistake 1

Using sorting.

Example:

```
nums.sort(key=lambda x: x % 2)
```

Works.

But:

$O(n \log n)$

Not optimal.

Mistake 2

Forgetting to move pointers after swap.

Can create:

Infinite Loop

Mistake 3

Checking:

```
num % 2 == 1
```

only.

Remember:

```
Even => num % 2 == 0
```

Edge Cases

Single Element

Input:

```
[5]
```

Output:

```
[5]
```

All Even



Input:

[2,4,6]

Output:

[2,4,6]

All Odd

Input:

[1,3,5]

Output:

[1,3,5]

12. Detailed Dry Run

Input:

nums = [3,1,2,4]

Left	Right	Array	Action
0	3	[3,1,2,4]	Swap
1	2	[4,1,2,3]	Swap
2	1	[4,2,1,3]	Stop

Return:

[4,2,1,3]

13. Common Use Cases (Real-Life / Interview)

Data Segregation

Group records into categories.

QuickSort Partitioning

Core partition idea.

Memory Efficient Rearrangement

In-place processing.

Interview Purpose

Tests:

Two Pointers

Partitioning



Swapping

14. Common Traps (Important!)

Trap 1

Thinking sorting is required.

Not needed.

Only partitioning is required.

Trap 2

Using extra arrays unnecessarily.

Trap 3

Not recognizing partition pattern.

Whenever you see:

Move category A left

Move category B right

Think:

Two Pointers

15. Builds To (Related LeetCode Problems)

LC 905 - Sort Array By Parity

↓

LC 283 - Move Zeroes

↓

LC 27 - Remove Element

↓

LC 75 - Sort Colors

↓

LC 977 - Squares of Sorted Array

These are classic partitioning problems.

16. Alternate Approaches + Comparison

Approach	Time	Space	Interview Rating
Extra Arrays	$O(n)$	$O(n)$	Good
Sorting	$O(n \log n)$	$O(1)$	Fair



Two Pointers	$O(n)$	$O(1)$	Best
--------------	--------	--------	------

Approach 1: Extra Arrays

even = []

odd = []

Simple.

But extra memory.

Approach 2: Sorting

nums.sort(key=lambda x: x % 2)

Easy.

Not optimal.

Approach 3: Two Pointers (Chosen)

Most efficient.

Interview favorite.

17. Why This Solution Works (Short Intuition)

The left pointer searches for misplaced odd numbers.

The right pointer searches for misplaced even numbers.

Whenever both are found, swapping fixes two positions at once, eventually partitioning the array into even and odd sections.

18. Variations / Follow-Up Questions**Follow-Up 1**

Keep relative order unchanged.

Related:

Stable Partition

Requires extra space.

Follow-Up 2

Sort by parity index.

Related:

LC 922

Sort Array By Parity II

Follow-Up 3



Partition positives and negatives.

Same pattern.

Follow-Up 4

Implement Dutch National Flag algorithm.

Related:

LC 75

Sort Colors

Interview Takeaway

This is a classic **Two Pointer Partitioning** problem.

Core idea:

Find misplaced element from left

Find misplaced element from right

Swap

Whenever you hear:

Even/Odd

Positive/Negative

Zero/Non-Zero

Category A/Category B

think:

Partitioning + Two Pointers

This pattern appears frequently in array manipulation and in-place optimization interview questions.